

Advanced RAG Architectures — Interview Questions

Reproduced exactly as originally written from Retrieval Augmented Generation (RAG)/Advanced RAG Architectures/Advanced RAG Architectures - Video Notes.md — *nothing reworded*.

Where the RAG Workflow set covers retrieval mechanics and the Current State of RAG set covers production failure modes, this set is about architectural pattern selection — the menu of modules you can plug into each stage of a RAG system, and the judgement of which ones a given problem actually needs. Curated from current (2026) advanced-RAG and search-engineering interview question banks, calibrated to roughly four years of AI engineering experience. Same two-layer format: a plain-language answer with an everyday example, then a crisp, technically precise version worth saying out loud in the room. Try answering out loud before reading the model answer.

 **Interview Q1:** "Before a query ever reaches the retriever, what can you do to it, and why would you bother?"

✓ **Strong answer:** "The user's question, exactly as typed, is usually a bad *search query* — and no amount of good retrieval downstream fixes a bad query going in. Query translation is the family of techniques that reshape the question first.

Everyday example: a real user types '*man that SBF blowup is crazy! What is LangChain?*' Fed straight into a retriever, the noise about Sam Bankman-Fried drags the embedding toward crypto documents, and the answer that comes back is vague enough that you can't even tell which question was being answered. Rewritten to '*What is the definition and purpose of LangChain?*', both retrieval and generation get dramatically better — same pipeline, same index, just a cleaner question.

There are five main moves, and they're not interchangeable:

User's raw question	Rewriting	→ one messy query	→ one clean query
	Multi-Query	→ one query	→ several rephrasings (fuse with RRF)
	Decomposition	→ one compound Q	→ several sub-questions (components)
	Step-Back	→ one narrow Q	→ a broader Q first, then the narrow one
	HyDE	→ one question	→ a hypothetical ANSWER, embedded instead

The reason it's worth the trouble is leverage: this is the cheapest stage to fix. A bad question poisons every stage after it, and an extra LLM call here costs far less than a wrong answer."

 **Standard Interview Answer:** "Query translation is the pre-retrieval stage that transforms the raw user input into one or more better-formed retrieval queries. The main techniques are query rewriting (denoise and clarify a single query), multi-query generation (fan out into several paraphrases, retrieve for each, fuse with RRF), query decomposition (split a compound question into independently-answerable sub-questions), step-back prompting (abstract to a more general question to establish context before the specific one), and HyDE (generate a hypothetical answer document and embed *that* instead of the question). All of them trade an additional LLM call and its latency for improved retrieval relevance — justified because query-side defects propagate irreversibly through every downstream stage."

 **Interview Q1 (follow-up):** "Rewriting, multi-query, and decomposition all sound like 'turn the question into different questions.' How do you actually choose between them?"

✓ **Strong answer:** "They fail on different shapes of input, which is the tell.

- **Rewriting** is for a query that's *noisy or malformed* — one question in there, just badly expressed. One in, one out.

- **Multi-query** is for a query that's *fine but narrow* — the phrasing is reasonable, but a single embedding might miss documents that say the same thing differently. One in, several paraphrases out, all searched, results fused.
- **Decomposition** is for a query that's *genuinely more than one question*. 'What's the return policy for electronics, and what's the warranty period?' — those are two separate retrievals, not two phrasings of one.

The quick diagnostic: ask whether the sub-queries you'd produce are *rephrasings* (→ multi-query) or *components* (→ decomposition). If neither, and the query is just messy, it's rewriting."

 **Standard Interview Answer:** "The selection criterion is the defect in the input query. Query rewriting addresses lexical noise and underspecification in a single-intent query — a one-to-one transformation. Multi-query generation addresses recall limitations from single-embedding sensitivity by producing semantically equivalent paraphrases whose result sets are fused, typically via RRF. Query decomposition addresses multi-intent queries, where sub-queries are semantically disjoint components requiring independent retrieval and a merge step. Applying decomposition to a single-intent query wastes retrieval calls; applying multi-query to a genuinely compound question under-serves at least one intent."

 **Interview Q2:** "Explain HyDE. Why would embedding a fabricated answer ever beat embedding the real question?"

 **Strong answer:** "Because a question and an answer don't *look* alike in embedding space, even when they're about exactly the same thing. Your index is full of answer-shaped text — passages, paragraphs, documentation. Your query is question-shaped. You're comparing two different genres of text and hoping they land close together.

HyDE closes that gap by having the LLM write a plausible *fake answer* to the question first, then embedding that instead. The fake answer is answer-shaped, so now you're comparing like with like.

Everyday example: asking 'What causes rust?' versus a hypothetical answer 'Rust forms when iron reacts with oxygen and water, producing hydrated iron oxide...' — that second text sits much closer in vector space to the real chemistry passage in your index, because it's written in the same register, with the same vocabulary, at the same length.

The catch, and it's a real one: the hypothetical document is generated with no grounding whatsoever. If the model invents specific details — a fake statistic, a fake product name — you're now searching the neighbourhood of a hallucination. HyDE tends to help most on general or conceptual questions where the model's prior is roughly right, and hurt most on narrow factual or proprietary-domain queries where it confidently invents specifics your corpus never contained."

 **Standard Interview Answer:** "HyDE — Hypothetical Document Embeddings — addresses query-document asymmetry: interrogative text and declarative passage text occupy systematically different regions of embedding space, so question-to-passage cosine similarity is a weaker signal than passage-to-passage similarity. HyDE prompts an LLM to generate a hypothetical answer document, embeds that, and uses it as the retrieval vector, converting the comparison into a document-to-document match. The trade-off is an added ungrounded generation step on the critical path: it adds latency and cost, and on domain-specific or fact-heavy queries the hypothetical document can hallucinate specifics that steer retrieval toward a wrong region of the index. It performs best where the base model has reasonable priors about the answer's *form*, even without knowing the specific content."

 **Interview Q3:** "What is step-back prompting, and how is it different from decomposition?"

✅ **Strong answer:** "Decomposition splits a question *sideways* into parts. Step-back moves *upward* to a more general question first.

Everyday example from the session: '*Who won the first prize in shooting at the Olympics?*' — searched directly, that can retrieve nothing useful, because it presupposes context the index isn't organised around: which Olympics, what year, were shooting events even held. A step-back approach first retrieves the broader context ('*when was the most recent Olympics, and which shooting events were held*'), and only then answers the narrow question standing on top of that.

DECOMPOSITION – split horizontally into components

"grounds for divorce AND filing procedure"

└─> "grounds for divorce"

└─> "filing procedure"

STEP-BACK – move vertically to a more general concept

"who won first prize in shooting at the Olympics?"

↑ first: "which Olympics, what shooting events were held"

↓ then: the specific winner, with that context in hand

Use step-back when the specific query is too concrete and skips over the governing concept the answer actually depends on. Use decomposition when the query genuinely contains more than one thing being asked."

 **Standard Interview Answer:** "Step-back prompting generates a more abstract formulation of the original query, retrieves against that abstraction to establish governing context, and then resolves the specific query with that context in scope. It differs from decomposition along the axis of transformation: decomposition partitions a multi-intent query into semantically disjoint sibling sub-queries, whereas step-back produces a hierarchically *superordinate* query. Step-back is indicated when a query is over-specified relative to how the corpus is organised — when the concept or scoping fact required to interpret the answer isn't present in the query itself — rather than when the query contains multiple intents."

 **Interview Q4:** "In RAG-Fusion you end up with several ranked lists. Why fuse them with RRF rather than just normalising the scores and averaging?"

✅ **Strong answer:** "Because the scores from different retrievers aren't the same kind of number, and no amount of rescaling really makes them one.

Everyday example: BM25 returns unbounded positive scores that depend on query length and term statistics. Cosine similarity is bounded roughly –1 to 1. Averaging those is like averaging a temperature in Celsius with a distance in miles — the arithmetic runs fine and the result is meaningless. Min-max rescaling looks like a fix, but a single outlier score compresses everything else into a narrow band, and both methods quietly assume that 'BM25 confidence at rank 1' and 'cosine confidence at rank 1' mean the same thing. They don't.

RRF sidesteps all of it by throwing the scores away and using only *position*:

$$\text{RRF_score}(d) = \sum_{\text{over every list } L \text{ that contains } d} 1 / (k + \text{rank_L}(d))$$

A rank position is the one universal language every ranker speaks — 1st means 1st regardless of what produced it. And it de-duplicates for free: a document retrieved by three of your fanned-out sub-queries accumulates score across all three lists and appears once in the output, instead of eating three slots of your context window."

🔗 **Standard Interview Answer:** "RRF is a rank-aggregation algorithm that combines multiple ranked lists using only each document's rank position, scoring each document as the sum over lists of $1/(k + \text{rank})$, with k a smoothing constant conventionally set to 60 after Cormack et al. (2009). It's preferred over score fusion because relevance scores from heterogeneous retrievers are not commensurable — BM25 produces unbounded, corpus-statistic-dependent scores while cosine similarity is bounded — and normalisation schemes like min-max are outlier-sensitive and still presuppose cross-retriever score comparability that doesn't hold. RRF requires no score normalisation, works when a retriever exposes no scores at all, and inherently de-duplicates documents appearing across multiple lists by accumulating their contributions."

💬 **Interview Q4 (follow-up):** "Is $k = 60$ just a magic number you'd accept, or would you tune it?"

✅ **Strong answer:** "I'd treat it as a hyperparameter, and I'd know *why* the default is often wrong for a RAG setup."

$k = 60$ came from a pilot study fusing many comparable TREC systems over deep result lists, and the original point of the constant was to stop one outlier system's high rankings from dominating the fusion. In that setting the optimum is genuinely flat — anywhere from about 20 to 100 barely moves the metric.

The problem is that a modern hybrid-search setup isn't that setting. With $k = 60$, the curve $1/(60 + \text{rank})$ is almost flat across the top 20 positions: being ranked #1 instead of #2 is worth about 1.6%, while merely *appearing* on a second list is worth roughly 15 rank positions. So RRF behaves as a consensus vote — it systematically prefers a document both retrievers found mediocre over a document one retriever found decisively excellent. When you're deliberately fusing two *complementary* retrievers (that's the whole point of hybrid search), penalising decisive single-retriever wins is exactly backwards.

What I'd actually do: tune k on a labelled set — expect an optimum well below 60 — and, more importantly, stop asking RRF to make the final call. Use it for candidate generation at depth 100+, then let a cross-encoder or LLM reranker decide the final ordering."

🔗 **Standard Interview Answer:** " k is a tunable smoothing constant, and the conventional default of 60 is a legacy of the original evaluation setting — fusing many comparable systems over deep TREC runs, where the MAP optimum is flat across roughly $k \in [20, 100]$. In a two-retriever hybrid-search configuration over shallow lists, $k = 60$ flattens the score curve across the top ranks such that cross-list agreement dominates single-list rank quality, biasing fusion toward consensus-mediocre documents over decisively-ranked ones — an anti-pattern when the retrievers were selected specifically for complementarity. The mitigations are to tune k against labelled relevance judgements rather than inheriting the default, and to scope RRF to candidate generation at sufficient depth while delegating final ordering to a cross-encoder or LLM reranker."

🗣️ **Interview Q5:** "RankGPT uses an LLM as the re-ranker. When would you actually reach for that over a dedicated cross-encoder reranker?"

✅ **Strong answer:** "Rarely, in production — and the reason is cost and latency, not quality."

Everyday example: it's the difference between having a subject-matter expert personally read every shortlisted resume versus using a fast screening model trained specifically to score resume-to-job relevance. The expert is more nuanced. The expert is also three orders of magnitude more expensive per decision, and you can't have them read a thousand.

Concretely, listwise reranking 20 candidates with a frontier LLM runs roughly **\$0.01–0.03 per query and adds about 2–5 seconds** of latency. A dedicated reranker sits around **130ms at p50**, with per-token cost roughly a thousandth of frontier-model pricing. There's also a hard structural limit: you can't fit a thousand candidates in one prompt, so listwise LLM reranking needs a sliding window — rerank 30 at a time, slide forward — which multiplies the calls and the latency again.

Where RankGPT genuinely earns its place: offline or batch scenarios — generating training labels to distil into a small reranker, building an evaluation gold set, or reranking a very small candidate list in a low-QPS, high-value workflow (legal or medical review) where 3 seconds and 2 cents are irrelevant next to being right. For anything user-facing and interactive, a dedicated cross-encoder is the default."

🔗 **Standard Interview Answer:** "RankGPT applies an LLM as a listwise reranker, prompting it to permute a candidate list by relevance rather than scoring pairs independently. It's competitive with dedicated cross-encoders on TREC-DL and BEIR, but the production economics are unfavourable: listwise LLM reranking of ~20 candidates incurs roughly \$0.01–0.03 per query and 2–5 seconds of added latency, versus approximately 130ms p50 and ~3 orders of magnitude lower cost for a specialised reranker, and context-length limits force sliding-window passes over larger candidate sets. The defensible use cases are offline: distillation-label generation for training a compact reranker, evaluation set construction, or low-QPS high-stakes review workflows. For interactive, latency-bound serving, a cross-encoder — or a distilled listwise model — is the standard choice."

🗣️ **Interview Q6:** "Walk me through two-stage RAG. Why is 'retrieve wide, then rerank narrow' better than just retrieving the top 3 directly?"

✅ **Strong answer:** "Because the two stages are optimising for different things, and you want both."

The retriever is built for **speed at scale** — it compares pre-computed vectors, so it can sweep millions of chunks fast, but it's comparing two vectors that were embedded independently, never together. The reranker is built for **judgement** — it reads the query and one document *jointly*, which is far more accurate, but far too slow to run over the whole corpus.

So you split the job: retriever maximises **recall** cheaply, reranker maximises **precision** expensively over a small survivor set.

```
1M chunks
|
| retriever (fast, approximate) – get RECALL right
|
▼
top 25 candidates
|
| reranker (slow, accurate) – get PRECISION right
|
▼
top 3 → LLM
```

The worked example from the session makes the point: three documents came back from vector search ranked 11, 9, 14. After reranking, the correct order was 14, 11, 9 — the document the vector search ranked *last* was actually the most relevant. At three documents that's a small difference. Retrieve ten and rerank, and documents 5 and 7 may be replaced outright by documents 22 and 30 — ones plain vector search had buried far down the list and you'd never have seen.

The critical constraint: reranking can only reorder what retrieval already found. If the right chunk isn't in the top 25, no reranker saves you — which is why the first stage is tuned for recall, deliberately over-fetching."

🔗 **Standard Interview Answer:** "Two-stage retrieval decouples recall optimisation from precision optimisation. The first stage uses a bi-encoder with an ANN index to fetch a deliberately over-sized candidate set (typically 20–100) at low latency, accepting imprecise ordering. The second stage applies a cross-encoder that jointly encodes query and candidate, producing substantially more accurate relevance estimates at a per-pair cost that would be prohibitive across the full corpus. The architecture is justified because cross-encoder quality doesn't scale to corpus-wide application and bi-encoder ordering is unreliable at the top of the list. The binding constraint is first-stage recall: reranking is order-preserving over the candidate set and cannot recover a relevant document absent from it, so first-stage K should be sized against a measured recall target rather than the LLM's context budget."

 **Interview Q7:** "Why would a RAG system need a router at all? Give me a concrete failure it prevents."

✅ **Strong answer:** "Because putting unrelated domains in one index lets them contaminate each other, and the retriever has no way to know it's happening."

The concrete failure from the session: build one vector store for school students covering physics, chemistry, maths and biology. Thermodynamics appears in *both* physics and chemistry, with different laws in each. A thermodynamics query retrieves a blend of both, and the model produces a confidently mixed-up answer. A sharper version: a particular formula appeared in both the physics document and the maths document — the retriever ranked the *maths* version first and never surfaced the physics one, so the student asking a physics question got the wrong formula with no indication anything went wrong.

The fix is one store per domain, with a router deciding which store a query goes to. A physics question only ever touches physics.

The other big driver is multi-tenancy. If you serve RAG to several banks, their data legally cannot share an index — each tenant gets its own store and the router enforces that boundary.

Two flavours: **logical routing** (a rule, a classifier, or an LLM picks the store by name — note that routing does *not* require an LLM; `if 'finance' in query` is a legitimate router) and **semantic routing** (embed the query, compare it against embedded prompt templates per domain, route to the nearest). Semantic is generally better, but on a specialised domain it may need a fine-tuned embedding model to separate the templates cleanly."

🌀 **Standard Interview Answer:** "Routing addresses cross-domain interference in a shared index: when semantically overlapping content from distinct domains coexists, similarity search cannot distinguish domain intent, producing blended or wrong-domain retrievals with no error signal. Partitioning by domain and placing a router upstream restores precision. Routing is also the enforcement mechanism for multi-tenant isolation, where per-tenant stores are a compliance requirement rather than an optimisation. Implementations are logical — deterministic rules, a text classifier, or an LLM selecting among described data sources — or semantic, embedding the query and selecting by similarity against per-domain prompt-template embeddings. Routing is a modular component, warranted when multiple domains or tenants exist and omitted entirely for a single-corpus system."

💬 **Interview Q7 (follow-up):** "Your router misclassifies a query. What happens, and how do you design around it?"

✅ **Strong answer:** "A misroute is worse than a bad retrieval, because it's silent and unrecoverable within the request. The query gets searched in a store that simply doesn't contain the answer, so the retriever returns whatever was *least bad* in the wrong domain — plausible, confidently ranked, and completely irrelevant. Nothing downstream can detect it: the reranker only reorders what it was given, and the generator has no way to know the right store was never consulted."

What I'd actually do: - Tier the routers. Deterministic rules first for unambiguous cases (tenant ID, explicit domain keywords) — those should never reach a model at all. Semantic routing for fast triage on the rest. - **Route on confidence, not just argmax.** If the top semantic match is below threshold — around 0.6 is a common starting point — don't commit. Fall back to an LLM router for a considered judgement, or fan out to the top-2 stores and let RRF merge the results, trading a little latency for not being catastrophically wrong. - **Log the routing decision explicitly.** It's the first thing you need when debugging a bad answer, and without it you'll waste time investigating retrieval quality in a store that was never the right one.

Reported gains from that tiered pattern are substantial — response times dropping from ~1.8s to ~0.5s with retrieval accuracy moving from ~72% to ~92% — because the cheap path handles most traffic and the expensive path only handles genuinely ambiguous queries."

 **Standard Interview Answer:** "Router misclassification is a silent, non-recoverable failure: the query is evaluated against a partition lacking the relevant content, and downstream stages have no signal that the wrong partition was searched — reranking is closed over the candidate set and generation is conditioned on whatever context it receives. Mitigations are architectural: a tiered router with deterministic pre-routing on unambiguous signals (tenant context, explicit domain markers), confidence-thresholded semantic routing with fallback to an LLM router or multi-store fan-out plus RRF merge when the top-match score is below threshold, and mandatory instrumentation of the routing decision for post-hoc attribution. The tiered pattern also improves aggregate latency, since the cheap deterministic and semantic paths absorb the majority of traffic and the expensive LLM path handles only low-confidence residue."

 **Interview Q8:** "CRAG, Self-RAG, and RRR all add feedback loops. What's each one actually checking, and where does it sit?"

 **Strong answer:** "They're the same idea — *don't trust this stage's output blindly, grade it* — applied at two different checkpoints.

```
Query → Retrieve → [CHECKPOINT 1: are these documents any good?] ← CRAG
                        → Generate → [CHECKPOINT 2: is this answer any good?] ← Self-RAG / RRR
                        → User
```

CRAG (Corrective RAG) grades the retrieved documents, before generation. A grader — an LLM, an embedding model, or a plain script — scores each document for relevance. If at least one passes, generate normally. If none do, *don't* return 'no context found' — correct course, most commonly by rerouting to a live web search, scraping the results, and generating from that instead. Web search is just the suggested correction; at that same decision point you could equally apply query rewriting or decomposition and retrieve again.

Everyday example: someone asks about a product released yesterday. The vector store has nothing recent, the grader flags every result as irrelevant, and the query falls through to web search rather than failing.

Self-RAG grades the generated answer, after generation. If the answer is unsupported or vague, feed it back with the original query, note that it failed, and loop — this time with more information than the first attempt had. Self-RAG's fuller form also decides whether retrieval is needed *at all* for a given query, and uses reflection tokens with a critic model rather than a single pass/fail.

RRR (Rewrite-Retrieve-Read) sits alongside Self-RAG in the same generation-side slot — it's the loop that acts on that verdict by rewriting the question and retrieving again, rather than just re-generating from the same context.

The unifying principle: you can place a feedback checkpoint after retrieval, after generation, or both — and if either check fails, the pipeline loops back with more context than it had before."

 **Standard Interview Answer:** "These are self-reflective RAG architectures differing by the stage they instrument. CRAG places an evaluator after retrieval that grades document relevance and triggers corrective action on failure — canonically a fallback to web search, though any query-translation technique can occupy that branch — preventing generation from proceeding on irrelevant context. Self-RAG instruments generation, critiquing the produced answer for support and relevance, and in its full formulation uses reflection tokens and a critic model to also decide whether retrieval is warranted per-query rather than unconditionally. RRR (Rewrite-Retrieve-Read) occupies the same generation-side slot, using generation quality as the signal to rewrite the query and re-retrieve rather than merely re-generate. Architecturally they're composable: checkpoints can be placed post-retrieval and post-generation simultaneously, each with its own corrective branch."

 **Interview Q8 (follow-up):** "These loops can run more than once. What stops them, and what's the cost?"

 **Strong answer:** "Nothing stops them automatically, which is exactly the risk — you need to build the stop.

Two things are required. First, a **grading script** that decides 'good enough' — and it can be an LLM, an embedding-similarity check, or a custom NLP script; it doesn't have to be expensive. Second, and non-negotiably, a **hard cap**: refine two or three times, then return the best attempt you have. Without the cap, a grader that never returns 'satisfied' loops forever, and the failure mode isn't a wrong answer — it's a request that never terminates.

The cost is multiplicative, not additive. Each loop is a full retrieve-plus-generate cycle, so a system that averages 800ms can hit 2.5s on its third attempt — and that shows up in p99, not the mean, so it hides from average-latency dashboards.

The subtler problem: the grader is itself a fallible model. A grader that's too strict burns loops on answers that were already fine; too lenient and it waves through exactly the failures it was added to catch. So the grader needs evaluating against ground truth like any other component — it's not a free oracle just because it's sitting in a judging role."

 **Standard Interview Answer:** "Termination requires an explicit policy: a relevance grader defining the satisfaction condition, plus a hard iteration cap — conventionally two to three refinement cycles — after which the best available attempt is returned. Absent the cap, a mis-calibrated grader produces unbounded loops. Cost scales multiplicatively, since each iteration is a complete retrieve-and-generate cycle, and the impact concentrates in tail latency rather than the mean, making it invisible to average-latency monitoring. Additionally, the grader is a model with its own error profile: over-strict grading wastes iterations on adequate outputs, over-lenient grading defeats the mechanism's purpose, so grader precision and recall require independent evaluation against labelled data rather than being assumed."

 **Interview Q9:** "Given this whole menu — query translation, routing, fusion, reranking, corrective loops — how do you decide what a specific system actually needs?"

 **Strong answer:** "The framing that helps most is that **RAG isn't one pipeline, it's a set of slots**. At every stage — question, routing, construction, indexing, retrieval, generation — there's a menu, and a real system fills only the few slots its actual problem needs. Attaching everything is how you end up with a 5-second p99 and no idea which module earned it.

My decision rule is to let each module be justified by a failure you can actually observe:

Attach this...	...only when you've observed
Routing	Multiple domains or tenants — cross-domain contamination, or a hard isolation requirement. A single-corpus system needs no router.
Query rewriting / HyDE	Real user queries that are noisy, or measurable question-to-passage mismatch
Decomposition	Genuinely compound questions under-served in production traffic
Multi-query + RRF	Recall misses where the right chunk existed but a single phrasing didn't surface it
Reranking	The right chunk lands in the candidate set but not in the top-K passed to the LLM
CRAG / Self-RAG	Answers generated confidently from irrelevant context, or measured unfaithfulness

The discipline is diagnosis before addition. Each module costs latency and money, and every one you attach is another stage that can fail — which compounds against you. Routing is genuinely optional. Reranking helps but isn't mandatory. The skill being tested isn't knowing the menu; it's knowing which slots this problem actually needs filled, and being able to say why the rest are absent."

🔗 **Standard Interview Answer:** "Advanced RAG is best modelled as a set of optional modules per pipeline stage rather than a canonical architecture, and module selection should be failure-driven: each addition justified by an observed, measured deficiency rather than adopted by default. Routing is warranted by multi-domain or multi-tenant partitioning requirements; query translation by demonstrated query-side defects (noise, asymmetry, multi-intent); multi-query fusion by recall gaps attributable to single-embedding sensitivity; reranking by candidate-set-present-but-top-K-absent failures; and self-reflective loops by measured groundedness failures. Each module contributes latency, cost, and an additional failure surface that compounds multiplicatively with the rest of the pipeline, so unjustified modules degrade both performance and reliability. The operative discipline is instrumenting the pipeline to localise failure to a stage, then attaching the module that addresses that specific stage."

What interviewers are really scoring for, across all of the above: - Whether you can distinguish techniques that *sound* similar (rewriting vs. multi-query vs. decomposition; step-back vs. decomposition; RRF vs. reranking) on the basis of *what input defect each one fixes*, not just by definition - Whether you volunteer the cost side unprompted — every module here is a latency and money trade, and naming the trade is what separates practitioner from reader - Whether you know which defaults are legacy artefacts worth questioning ($k = 60$) rather than settled truth - Whether you treat graders, routers and rerankers as fallible models needing their own evaluation, rather than as oracles - Whether "which of these would you use" gets a *diagnostic method* — attach the module that fixes an observed failure — instead of an enthusiastic list of everything you've read about

Sources consulted while calibrating this section: - [Top 20 Advanced RAG & Search Engineering Interview Questions: 2026 — TechInterview](#) - [RAG Interview Questions and Answers Hub — KalyanKS-NLP \(GitHub\)](#) - [Which Query Transformation Techniques Actually Help RAG? — Alex Chernysh](#) - [RAG Architecture 2026: Patterns, Code, and Eval — FutureAGI](#) - [Reciprocal Rank Fusion: Why \$k=60\$ Buries Your Best Hit — DEV Community](#) - [Reciprocal Rank Fusion \(RRF\): How It Works and When to Use It — BigData Boutique](#) - [Should You Use LLMs for Reranking? Pointwise, Listwise, and Cross-Encoders — ZeroEntropy](#) - [Listwise Reranking: LLM Permutation Over a Candidate List — ZeroEntropy](#) - [Cross-Encoders, ColBERT, and LLM-Based Re-Rankers: A Practical Guide — Michael Ryaboy](#) - [Self-Reflective RAG with LangGraph — LangChain](#) - [Corrective RAG \(CRAG\): Workflow, Implementation, and More — Meiliseach](#) - [Logical & Semantic Query Routing in RAG Apps — Towards Data Science](#) - [RAG Query Routing in Practice: Multi-Vector Store Coordination — BetterLink Blog](#) - [RAG Interview Questions \(2026\): The Complete Guide — GitGood](#) - [Top 30 RAG Interview Questions and Answers for 2026 — DataCamp](#)